

Entity Types :

1. If an object has its own database identity (primary key value) then its type is Entity Type.
2. An entity has its own lifecycle. It may exist independently of any other entity.
3. An object reference to an entity instance is persisted as a reference in the database (a foreign key value).

eg : College is an Entity Type. It has its own database identity (It has primary key).

4. Mandatory annotations -- @Entity + @Id

Value Types :

1. If an object doesn't have its own database identity (no primary key value) then its type is Value Type.
2. Value Type object belongs to an Entity Type Object.
3. It is embedded in the owning entity and it represents the table column in the database.
4. The lifespan of a value type instance is bounded by the lifespan of the owning entity instance.
5. Annotation -- @Embeddable

Different types of Value Types

Basic, Composite, Collection Value Types :

1. Basic Value Types :

Basic value types are : they map a single database value (column) to a single, non-aggregated Java type.

Hibernate provides a number of built-in basic types.

String, Character, Boolean, Integer, Long, Byte, etc.

- 2.

Composite Value Types :

In JPA composite types also called Embedded Types. Hibernate traditionally called them Components.

- 2.1 Composite Value type looks like exactly an Entity, but does not own lifecycle and identifier.

Annotations Used

1. @Embeddable :

Defines a class whose instances are stored as an intrinsic part of an owning entity and share the identity of the entity. Each of the persistent properties or fields of the embedded object is mapped to the database table for the entity. It doesn't have own identifier.

eg : Address is eg of Embeddable

Student HAS-A Address (eg of Composition --i.e Address can't exist w/o its owning Entity i.e Student)

College HAS-A Address (eg of Composition --i.e Address can't exist w/o its owning Entity i.e College)

BUT Student will have its own copy of Address & so will College (i.e Value Types don't support shared reference)

2. @Embedded :

Specifies a persistent field or property of an entity whose value is an instance of an embeddable class. The embeddable class must be annotated as Embeddable.

eg : Address is embedded in College and User Objects.

3. @AttributeOverride :

Used to override the mapping of a Basic (whether explicit or default) property or field or Id property or field.

In Database tables observe the column names. Student table having STREET_ADDRESS column and College table having STREET column. These two columns should map with same Address field streetAddress. @AttributeOverride gives solution for this.

To override multiple column names for the same field use @AttributeOverrides annotation.

eg : In Student class :

@Embedded

@AttributeOverride(name="streetAddress", column=@Column(name="STREET_ADDRESS"))

private Address address;

where , name --POJO property name in Address class

3.

Collection Value Types :

Hibernate allows to persist collections.

But Collection value Types can be either collection of Basic value types, Composite types and custom types.

eg :

Collection mapping means mapping group of values to the single field or property. But we can't store list of values in single table column in database. It has to be done in a separate table.

eg : Collection of embeddables

@ElementCollection

@CollectionTable(name="CONTACT_ADDRESS", joinColumns=@JoinColumn(name="USER_ID"))

@AttributeOverride(name="streetAddress", column=@Column(name="STREET_ADDRESS"))

private List<ContactAddress> address;

eg : collection of basic type

@ElementCollection

@CollectionTable(name="Contacts", joinColumns=@JoinColumn(name="ID"))

@Column(name="CONTACT_NO")

private Collection<String> contacts;